The Design and Implementation of a Library-based Game Engine using C++

Problems

- Although popular game engines are powerful, they are too individualized with their ridiculous amount of editors and their own formats
- Some people prefer a code-based approach rather than an editor-based approach in many aspects of game development
- Game engines are not “portable” (their games require their own editors to continue making)
- I want to learn more about game engines and C++

About this Project

Existing solutions:

- Existing “solutions” include Raylib and Monogame, though Raylib wasn’t exactly designed for professional use and Monogame has virtually no support for 3d

Purpose:

- This project aims to provide users with a different approach to game development, using a code-first approach fully integratable within any IDE in the form of a library
- The library will be programmed using C++ and will have aspects including cross-platform, pipeline (graphics api) selection, and different control layers

About:

- The project is named “clfe” for “C Library Framework Engine”. The name was chosen very early so may not fit very well, but it can definitely be reinterpreted to match the current project
- The math library was designed from scratch instead of using a 3rd party library
- There are 3 main sections of the engine:
  - “clfe” with all functional components
  - “clm” for “C Library Math”, the math library
  - “clu” for “C Library Utilities”
- Visual Studio is the main IDE used. Although not yet compatible with cross-platform compilers, there has been an intentional avoiding of MSVC-specific functionalities

Design

Component: Math Library

Includes generic functions such as sin, sqrt, etc and vector and matrix math, implemented through templates. Vectors and matrices are critical to a 3d graphics system. Non-member operator overloads allow for better organization and broader utilizations.

```
template <msize_t Size, typename T, typename U>
    requires Arithmetic<T> && Arithmetic<U>
auto operator+(const Vector<Size, T>& vec1, const Vector<Size, U>& vec2)
```

Figure 1. Implicit template instantiation allows two Vectors of similar size but different types to be added together so long as the two types are compatible (ex. double + int = double)

InstanceList and the intrusive linked list model

The InstanceList is used by the System to index components. An intrusive linked list design is used to save on memory and performance overhead,

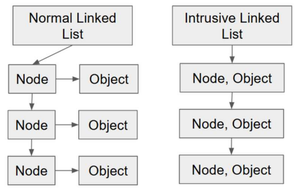

Figure 2.
The difference between a normal linked list and an intrusive linked list

SharedLink

The Shared Link was designed to handle component instance relationships. It mimics the intrusive linked list design by symbolizing the existence of a relationship between two component instances. Its core structure includes a link creator, the Well, and a link receiver, the Pool.

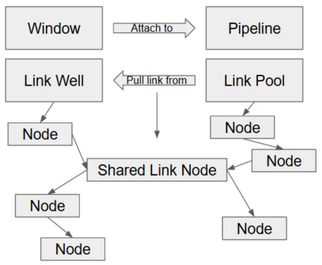

Figure 3.
A Shared Link acts as a node within both a Link Well and a Link Pool, simplifying implementation through an “intrusive” design. Upon creation or deletion, the Shared Link triggers mutual functions between the instances holding the Well and the Pool, resulting in the clean handling of initialization or deletion.

Through its design, a Well holding object can attach to multiple Pool holding objects and vice versa. The deletion of any Well or Pool holding objects triggers the deletion of all Shared Links it holds, subsequently triggering the handling of deletion between all linked objects.

Testing of the Engine

The engine is not yet at the stage where its metrics can be compared to other game engines for useful results. Basic performance checks, however, are still done to make sure nothing is horribly wrong.

Generic Game Engine Components

- **Window** - each instance represents an open window on the respective system.
- **Input** - input interface connected to windows. Can mostly be treated as part of a window.
- **Pipeline** - embodiment of different graphics apis, supported platforms depend on api and implementation.
- **System** - central indexing for larger component instances, such as windows and pipelines. Does not index small components such as 3d objects.
- **Scene** - specially designed to store 3d objects. Handed to a pipeline for rendering.

UniString

Created to solve the issues relating to wide and narrow strings within the engine. The UniString was designed to embody many string types as one, using narrow strings as a common interface.

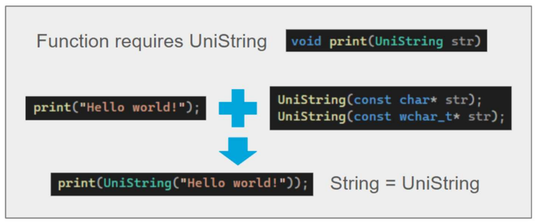

Figure 4. Constructors allow strings to be implicitly converted to UniString, making for seamless usage and integration into previously existing components

Pipeline: OpenGL 4.6

The first pipeline of the engine. OpenGL was chosen since it was the easiest api to set up, making for the perfect fast check for whether or not the engine is working. OpenGL 3.3 will definitely also be added in the future for compatibility purposes along with other classic apis such as Vulkan and DirectX.

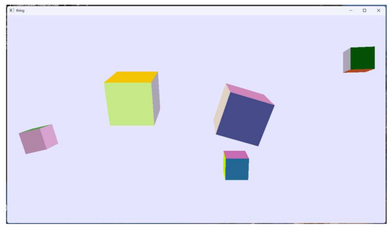

Figure 5.
A simple scene of 5 spinning cubes drawn using OpenGL 4.6

```
delta time: 0.0009057
fps: 1104.12
delta time: 0.0008333
fps: 1200.05
```

Figure 6.
The crazy fps resulting from the drawing of a super-simple scene (shown in Figure 5)

Results and Conclusions

Results

- 75+ code files
- 7000+ lines of code
- Doesn’t match the design goal perfectly but is on track towards the ultimate form of the design goal
- Has a solid foundation with a lot of room for new components and other improvements
- Currently has a OpenGL 4.6 pipeline, may have a Vulkan one by time of actual symposium
- As the main star of game engines, the 3d graphics will be continued to be developed
- Can be used as valuable insight for others who want to develop their own game engines

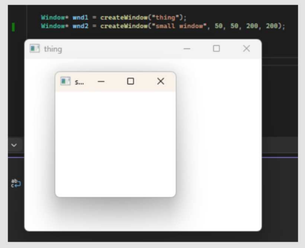

Figure 7. The creation of two windows with only two lines of code

Conclusions

- Though the visually seen capabilities of the engine may seem lacking, a very solid and large foundation is required for even the most basic 3d graphics, proving the extensive work underlying the engine
- The engine still aligns with the goal of being portable to any C++ supporting IDE (usually in the form of a library)
- The project is currently more useful as insight rather than in game development, but that will change as more and more components are developed

References

- Ma, L. (2026). C Library Framework Engine Planning and Documentation Document. Google Slides. C Library Framework Engine Planning and Documentation Document
- Unity Technologies. (2025). Unity - Manual: Rendering in the Universal Render Pipeline. Unity3d.com. https://docs.unity3d.com/6000.2/Documentation/Manual/urp/rendering-in-universalrp.html
- Vohera, C., Chheda, H., Chouhan, D., Desai, A., & Jain, V. (2021, July 1). Game Engine Architecture and Comparative Study of Different Game Engines. IEEE Xplore. https://doi.org/10.1109/ICCCNT51525.2021.9579618
- Sobota, B., & Pietrikova, E. (2023, August). The Role of Game Engines in Game Development and Teaching. ResearchGate. https://www.researchgate.net/publication/373282820_The_Role_of_Game_Engines_in_Game_Development_and_Teaching